

MERN stack - Shopping Online

Lab 01

Contents

1	Preparation	2
1.1	MongoDB Atlas	2
1.2	Hotmail	2
2	MERN stack	3
2.1	Setting up	3
2.1.1	Installation	3
2.1.2	Projects	3

1 Preparation

1.1 MongoDB Atlas

- Sign up an account from MongoDB Atlas: <https://www.mongodb.com/atlas/database>
 - Menu Projects ⇒ New Project
 - Name your project: <project_name>
 - Menu Database ⇒ Create a free database
 - Cloud provider & Region: AWS + Singapore
 - Cluster name: <cluster>
 - Menu Database Access ⇒ Create a Database User
 - Authentication method: Username and Password
 - Password authentication: <db_user> and <db_pass>
 - Menu Network Access ⇒ Add an IP address
 - Access list entry: 0.0.0.0/0 (allow access from anywhere)
- Download and install the MongoDB Compass (GUI) from: <https://www.mongodb.com/try/download/compass>
 - New connection ⇒ copy the connection string:
mongodb+srv://<db_user>:<db_pass>@<cluster>.mongodb.net/test
 - Create database: shoppingonline
 - Create collection: admins
⇒ Import JSON file from: <https://github.com/tsonkk/shoppingonline-resources/blob/main/mongodb/admins.json>
 - Create collection: categories
⇒ Import JSON file from: <https://github.com/tsonkk/shoppingonline-resources/blob/main/mongodb/categories.json>
 - Create collection: products
⇒ Import JSON file from: <https://github.com/tsonkk/shoppingonline-resources/blob/main/mongodb/products.json>

1.2 Hotmail

- Sign up an account from Microsoft: <https://signup.live.com>
 - New email: <email_user>@hotmail.com
 - Create password: <email_pass>
- Security tab: turn off the two-step verification
- Outlook application: review the welcome email
- SMTP settings from: <https://support.microsoft.com/en-us/office/pop-imap-and-smtp-settings-8361e398>
 - Server: smtp.office365.com
 - Port: 587
 - Encryption: STARTTLS
- Test an SMTP server:

1 `$ telnet smtp.office365.com 587`

2 MERN stack

2.1 Setting up

2.1.1 Installation

2.1.1.1 NodeJS

- Download and install the NodeJS from: <https://nodejs.org/en/download>

```
1 $ node --version
2 $ npm --version
```

2.1.1.2 Nodemon tool

- Install the Nodemon tool (use the sudo command for MacOS)

```
1 $ npm install nodemon --global
2 $ nodemon --version
```

2.1.1.3 ReactJS

- Installing create-react-app tool (use the sudo command for MacOS)

```
1 $ npm install create-react-app --global
2 $ create-react-app --version
```

2.1.1.4 Visual Studio Code IDE

- Download and install the Visual Studio Code from: <https://code.visualstudio.com/download>

2.1.2 Projects

```
1 |-- projectname
2   |-- server
3   |-- client-admin
4   |-- client-customer
```

2.1.2.1 Server project

- Create server project

```
1 <server>$ npm init -y
```

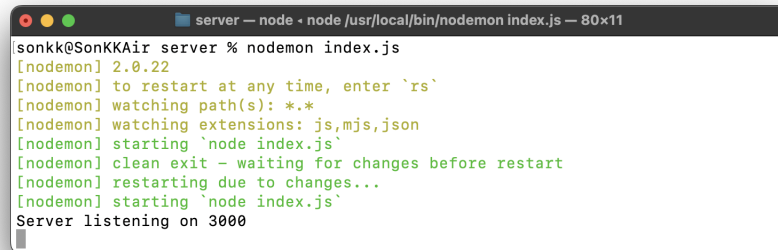
- Create 'server/index.js' file

```
1 //CLI: npm install express body-parser --save
2 const express = require('express');
3 const app = express();
4 const PORT = process.env.PORT || 3000;
5 app.listen(PORT, () => {
6   console.log(`Server listening on ${PORT}`);
7 });
8 // middlewares
9 const bodyParser = require('body-parser');
10 app.use(bodyParser.json({ limit: '10mb' }));
11 app.use(bodyParser.urlencoded({ extended: true, limit: '10mb' }));
12 // apis
13 app.get('/hello', (req, res) => {
14   res.json({ message: 'Hello from server!' });
15 });
```

- Start server

```
1 <server>$ nodemon index.js
```

- Postman: (GET) <http://localhost:3000/hello>



```
sonkk@SonKKAir server % nodemon index.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
[nodemon] clean exit - waiting for changes before restart
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Server listening on 3000
```

2.1.2.2 Client-admin project

- Create client-admin project

```
1 <projectname>$ npx create-react-app client-admin
```

- Update 'client-admin/package.json' file

```
1 {
2   ...,
3   "homepage": "/admin",
4   "proxy": "http://localhost:3000"
5 }
```

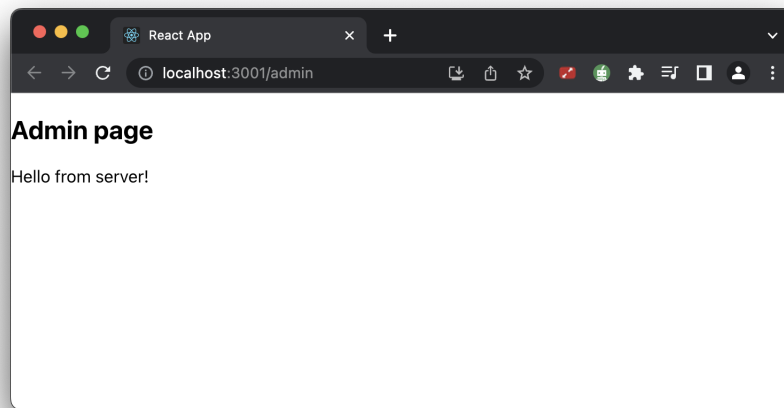
- Update 'client-admin/src/App.js' file

```
1 //CLI: npm install axios --save
2 import axios from 'axios';
3 import React, { Component } from 'react';
4
5 class App extends Component {
6   constructor(props) {
7     super(props);
8     this.state = {
9       message: 'Loading...'
10    };
11  }
12  render() {
13    return (
14      <div>
15        <h2>Admin page</h2>
16        <p>{this.state.message}</p>
17      </div>
18    );
19  }
20  componentDidMount() {
21    axios.get('/hello').then((res) => {
22      const result = res.data;
23      this.setState({ message: result.message });
24    });
25  }
26 }
27 export default App;
```

- Start client-admin

```
1 <client-admin>$ npm start
```

- Browser: <http://localhost:3001/admin>



2.1.2.3 Client-customer project

- Create client-customer project

```
1 <projectname>$ npx create-react-app client-customer
```

- Update 'client-customer/package.json' file

```
1 {  
2   ...,  
3   "homepage": "/",  
4   "proxy": "http://localhost:3000"  
5 }
```

- Update 'client-customer/src/App.js' file

```
1 //CLI: npm install axios --save  
2 import axios from 'axios';  
3 import React, { Component } from 'react';  
4  
5 class App extends Component {  
6   constructor(props) {  
7     super(props);  
8     this.state = {  
9       message: 'Loading...'  
10    };  
11  }  
12  render() {  
13    return (  
14      <div>  
15        <h2>Customer page</h2>  
16        <p>{this.state.message}</p>  
17      </div>  
18    );  
19  }  
20  componentDidMount() {  
21    axios.get('/hello').then((res) => {  
22      const result = res.data;  
23      this.setState({ message: result.message });  
24    });  
25  }  
26 }  
27 export default App;
```

- Start client-customer

```
1 <client-customer>$ npm start
```

- Browser: http://localhost:3002

